

RETAIL DATA INTEGRATION PLATFORM

Azure Data Factory — Complete Beginner-Friendly Project Report

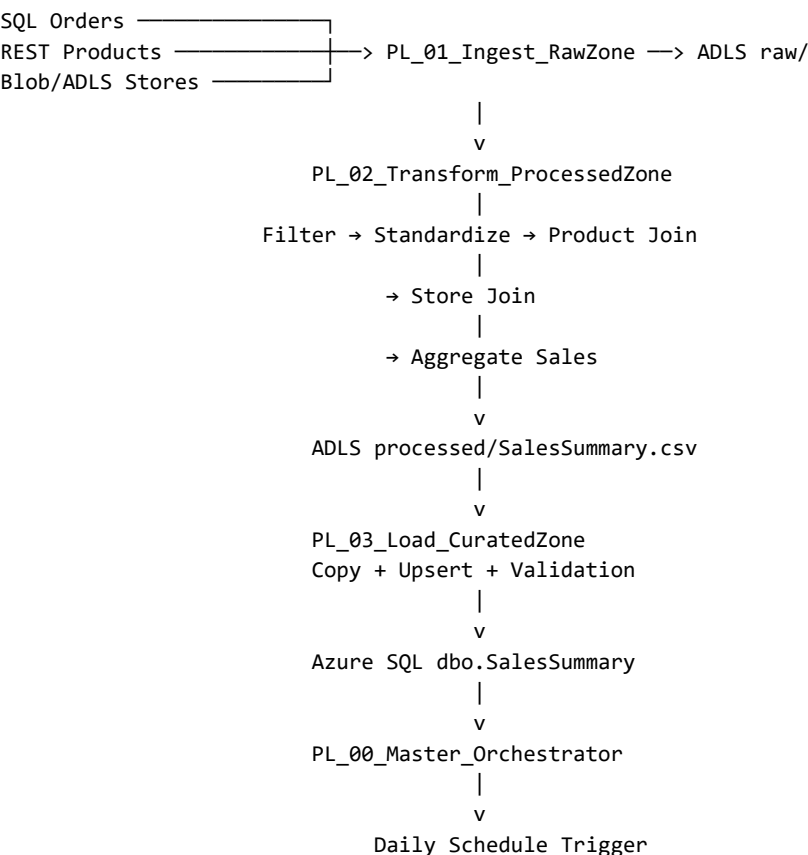
Implementation, configuration, testing, troubleshooting and deployment guide

1. Project Overview

This project implements an end-to-end retail data integration platform using Azure Data Factory. Orders are ingested from Azure SQL, Products from a REST/JSON source, and Stores from ADLS/Blob storage. The data is landed in a raw zone, transformed and joined with Mapping Data Flow, written to a processed SalesSummary.csv file, and finally loaded into Azure SQL. The solution includes Upsert loading, row-count validation, a master orchestrator and a daily schedule trigger.

This report combines the screenshots supplied in the uploaded 8-page document with the step-by-step actions that were completed during the build but were not necessarily visible in the screenshots.

2. Final Architecture



3. Azure Resources

Resource	Name	Purpose
----------	------	---------

Resource Group	Evan_DE	Azure resource container
Storage Account	evande	ADLS Gen2 storage
Containers	raw / processed	Raw and processed zones
Data Factory	evan-df	ETL and orchestration
SQL Server	evanserver	Azure SQL logical server
SQL Database	evandb	Source/control/curated tables

4. Step-by-Step Build

4.1 Create Resource Group

1. Open Azure Portal → Resource groups → Create.
2. Select your subscription.
3. Resource group: Evan_DE.
4. Choose a region.
5. Click Review + create → Create.

4.2 Create ADLS Gen2 Storage

6. Open Storage accounts → Create.
7. Use resource group Evan_DE.
8. Storage account name: evande.
9. Use standard performance and the low-cost redundancy suitable for the project.
10. Enable Hierarchical namespace for ADLS Gen2.
11. Review + create.
12. Open evande → Containers.
13. Create raw.
14. Create processed.

The supplied screenshots show the Azure resources, raw container and processed container on pages 1–4.
[filecite?turn2file0?L2-L5?](#)

4.3 Create Azure SQL

15. Create SQL logical server evanserver.
16. Create database evandb.
17. Use an affordable DTU-based tier for a student/demo project rather than Hyperscale.
18. Open SQL server → Networking.
19. Add your current client IP.
20. Enable Azure services access if required by the environment.
21. Open evandb → Query editor.

4.4 Create SQL Tables

```
CREATE TABLE dbo.Orders
(
    OrderID INT NOT NULL,
```

```

        OrderDate DATETIME NOT NULL,
        ProductID INT NOT NULL,
        StoreID INT NOT NULL,
        Quantity INT NOT NULL,
        UnitPrice DECIMAL(18,2) NOT NULL,
        OrderStatus VARCHAR(50) NULL,
        CONSTRAINT PK_Orders PRIMARY KEY (OrderID)
    );

CREATE TABLE dbo.WatermarkControl
(
    SourceID INT IDENTITY(1,1) PRIMARY KEY,
    SourceName VARCHAR(100) NOT NULL,
    SourceType VARCHAR(20) NOT NULL,
    LastLoadTimestamp DATETIME2 NULL,
    IsActive BIT NOT NULL DEFAULT 1
);

CREATE TABLE dbo.SalesSummary
(
    SalesDate DATE NOT NULL,
    StoreID INT NOT NULL,
    StoreName VARCHAR(200) NULL,
    ProductID INT NOT NULL,
    ProductName VARCHAR(200) NULL,
    Category VARCHAR(100) NULL,
    Ordercount INT NULL,
    TotalQuantity INT NULL,
    TotalSales DECIMAL(18,2) NULL,
    CONSTRAINT PK_SalesSummary PRIMARY KEY (SalesDate, StoreID, ProductID)
);

INSERT INTO dbo.WatermarkControl
(SourceName, SourceType, LastLoadTimestamp, IsActive)
VALUES
('Orders', 'SQL', '1900-01-01', 1),
('Products', 'REST', '1900-01-01', 1),
('Stores', 'Blob', '1900-01-01', 1);

```

The supplied SQL screenshot shows the Orders data and SQL Query Editor on page 4.

[filecite?turn2file0?L5-L5?](#)

4.5 Create Linked Services

22. ADF → Manage → Linked services → New.
23. Create AzureDataLakeStorage1 for storage account evande.
24. Create LS_SQL_SOURCE for evandb.
25. Create LS_SQL_TARGET for evandb.
26. Create RestService1 for the Products JSON source.
27. Use anonymous authentication for the public REST source.
28. Test each connection.

Linked services are shown in the supplied document on page 5. [filecite?turn2file0?L6-L6?](#)

4.6 Create Datasets

- DS_SQL_Orders → dbo.Orders.
- DS_REST_Products → Products.json through RestService1.

- DS_BLOB_Stores → Stores.csv.
- DS_RAW_Orders → raw/dbo.Orders.csv.
- DS_RAW_Products → raw/Products.csv.
- DS_PROCESSED_SalesSummary → processed output.
- DS_SQL_SalesSummary → dbo.SalesSummary.

4.7 Prepare/Verify Source Files

29. Open storage → raw.
30. Verify Stores.csv.
31. Verify dbo.Orders.csv after ingestion.
32. Verify Products.json is accessible from the REST source.
33. Do not manually modify the raw files just to make joins work.

Products.csv, Products.json, Stores.csv and dbo.Orders.csv are shown on pages 2–3. [filecite turn2file0 L3-L4](#)

4.8 Build PL_01_Ingest_RawZone

34. Create pipeline PL_01_Ingest_RawZone.
35. Add Lookup_ActiveSources.
36. Query active records from dbo.WatermarkControl.
37. Add ForEach_Source with @activity('Lookup_ActiveSources').output.value.
38. Add Switch_SourceType using @item().SourceType.
39. Create SQL, REST and Blob cases.
40. SQL case: Copy DS_SQL_Orders → DS_RAW_Orders.
41. After successful SQL copy, update the Orders watermark.
42. REST case: Copy DS_REST_Products → DS_RAW_Products.
43. Blob case: Get Metadata for Stores.csv and check Exists.
44. Validate and debug the pipeline.

4.9 REST Product Mapping — Important Fix

During the build, two mappings caused problems: [0] returned only the first product, and using a root-level \$ path for fields inside the selected array produced blank values. The working configuration is:

Collection reference:

```
$['products']
```

ProductID:

```
['ProductID']
```

ProductName:

```
['ProductName']
```

Category:

```
['Category']
```

UnitPrice:

```
['UnitPrice']
```

4.10 Build PL_02_Transform_ProcessedZone

45. Create PL_02_Transform_ProcessedZone.
46. Create Mapping Data Flow DF_Transform_ProcessedZone.

47. Add SourceOrders from DS_RAW_Orders.
48. Add SourceProducts from DS_RAW_Products.
49. Add SourceStores from DS_BLOB_Stores.
50. Add FilterValidRecords for required non-null keys/date.
51. Add DerivedStandardize for date/key standardization.
52. Join Products to Orders.
53. Join Stores using StoreID.
54. Add AggregateSales.
55. Add SinkProcessed.

The supplied screenshots show the Mapping Data Flow and successful transformation on pages 7–8.

[filecite?turn2file0?L8-L9?](#)

4.11 Product ID Normalization — Critical

The sample Orders data uses numeric ProductIDs such as 101–110, while the Products source uses P001–P025. This mismatch initially made JoinProducts return zero rows. The working data flow created/used a normalized numeric product key before the join. Keep the raw source unchanged and handle normalization in the transformation.

4.12 AggregateSales

- Group by SalesDate.
- Group by StoreID.
- Group by normalized ProductID.
- Group by StoreName.
- Group by ProductName.
- Group by Category.
- TotalQuantity = sum(toInteger(Quantity)).
- TotalSales = the configured quantity × unit-price sales expression.
- Ordercount = count().

The completed aggregate preview showed 10 rows.

4.13 Configure SinkProcessed

56. Select SinkProcessed.
57. Dataset: DS_PROCESSED_SalesSummary.
58. Map all nine output columns.
59. Open Settings.
60. File name option: Output to single file.
61. File name: SalesSummary.csv.
62. Keep the processed container/path.
63. Debug the data flow.
64. Confirm SalesSummary.csv contains 10 data rows.

The supplied document shows the processed SalesSummary.csv with 10 rows on page 4.

[filecite?turn2file0?L5-L5?](#)

4.14 Build PL_03_Load_CuratedZone

65. Create PL_03_Load_CuratedZone.

66. Add Copy_Processed_To_SQL.
67. Source: DS_PROCESSED_SalesSummary.
68. Sink: DS_SQL_SalesSummary.
69. Target table: dbo.SalesSummary.
70. Map all nine columns.
71. Set write behavior to Upsert.
72. Use SalesDate + StoreID + ProductID as the key.

An initial Insert run produced a duplicate primary-key error. Changing the sink to Upsert resolved the rerun problem and makes the load idempotent for the configured key.

4.15 Add Row-Count Validation

73. After Copy_Processed_To_SQL, add Lookup_SalesSummary_Count.
74. Use an SQL dataset connected to evandb.
75. Enable Use query and First row only.
76. Query: SELECT COUNT(*) AS TotalRows FROM dbo.SalesSummary;
77. Connect Copy success → Lookup success.
78. Add If_Rows_Exist.
79. Check whether TotalRows is greater than 0.
80. True: add Validation_Success Wait activity for 1 second.
81. False: add Fail_No_Rows.
82. Use an error code such as VALIDATION_FAILED and a message explaining that no rows were loaded.

The validation branch is visible in the supplied screenshots on pages 7–8. [filecite?turn2file0?L8-L9](#)

4.16 Build PL_00_Master_Orchestrator

83. Create PL_00_Master_Orchestrator.
84. Add Execute Pipeline → PL_01_Ingest_RawZone.
85. Enable Wait on completion.
86. Add Execute Pipeline → PL_02_Transform_ProcessedZone.
87. Connect PL_01 success → PL_02.
88. Add Execute Pipeline → PL_03_Load_CuratedZone.
89. Connect PL_02 success → PL_03.
90. Enable Wait on completion for each child pipeline.
91. Validate and debug.

The supplied screenshots show successful child pipeline runs and the master flow on pages 6–7. [filecite?turn2file0?L6-L8](#)

4.17 Create Daily Trigger

92. Open PL_00_Master_Orchestrator.
93. Add trigger → New/Edit → New.
94. Choose Schedule.
95. Name: TR_Daily_Master.
96. Frequency: Day; interval: 1.
97. Choose the desired execution time.
98. For an India-based schedule, use India Standard Time.
99. Apply and publish.

100. Monitor → Trigger runs.
101. Confirm TR_Daily_Master is enabled.

The trigger configuration is shown on page 8. [filecite?turn2file0?L9-L9](#)

5. End-to-End Test

102. Run PL_01 and verify raw files.
103. Run PL_02 and refresh JoinProducts, JoinStores and AggregateSales previews.
104. Confirm AggregateSales shows 10 rows for the sample.
105. Open processed and verify SalesSummary.csv.
106. Run PL_03 and verify dbo.SalesSummary.
107. Run PL_03 again to verify Upsert does not create duplicate-key failures.
108. Run PL_00_Master_Orchestrator and confirm PL_01, PL_02 and PL_03 all succeed.
109. Verify the daily trigger is enabled.

6. Troubleshooting

Problem	Fix
Products only has one row	Remove [0] and use collection reference <code>['products']</code> .
Products columns are blank	Inside the selected array use ['ProductID'], ['ProductName'], ['Category'], ['UnitPrice'].
JoinProducts has 0 rows	Normalize P001-style Product IDs to the numeric key used by Orders.
JoinStores type mismatch	Make both StoreID columns the same numeric type.
PathNotFound for processed/SalesSummary.csv	Set Sink Settings → Output to single file → SalesSummary.csv.
Duplicate PK in dbo.SalesSummary	Use Upsert with SalesDate, StoreID and ProductID as keys.
SQL connection failure	Check SQL networking, IP access, Azure services access and linked-service credentials.
Validation query error	Use <code>SELECT COUNT(*) AS TotalRows FROM dbo.SalesSummary;</code> with First row only enabled.

7. Final Output

- Raw zone: Products.csv, Stores.csv, dbo.Orders.csv.
- Processed zone: SalesSummary.csv.
- SQL target: dbo.SalesSummary.
- Sample transformation output: 10 rows.

- Master pipeline: PL_01 → PL_02 → PL_03.
- Automation: daily schedule trigger.
- Load behavior: Upsert.
- Validation: SQL row-count check.

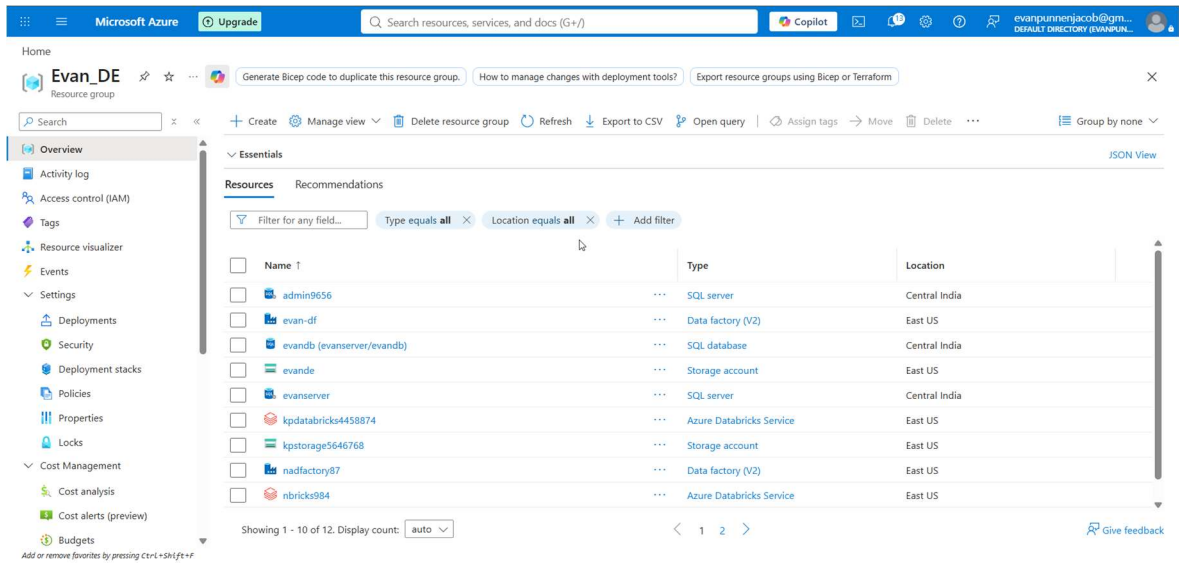
8. Final Checklist

- ☐ Azure resources created
- ☐ SQL tables created
- ☐ Linked services tested
- ☐ Datasets created
- ☐ PL_01 ingestion completed
- ☐ PL_02 transformation completed
- ☐ Product ID normalization completed
- ☐ Sales aggregation completed
- ☐ SalesSummary.csv generated
- ☐ SQL Upsert configured
- ☐ Row-count validation configured
- ☐ Master orchestrator completed
- ☐ Daily trigger enabled
- ☐ Project published
- ☐ End-to-end run succeeded

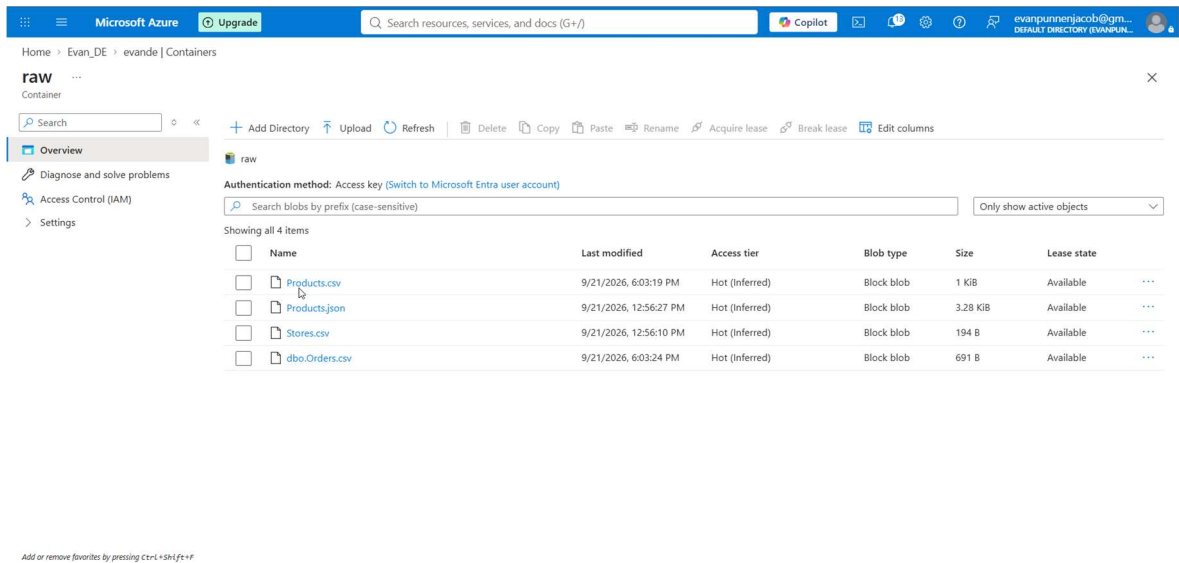
9. Supplied Screenshot Evidence

The uploaded document contains 8 pages of screenshots. They document resource setup, source files, processed output, SQL, linked services, pipeline runs, data flow, validation and trigger configuration.

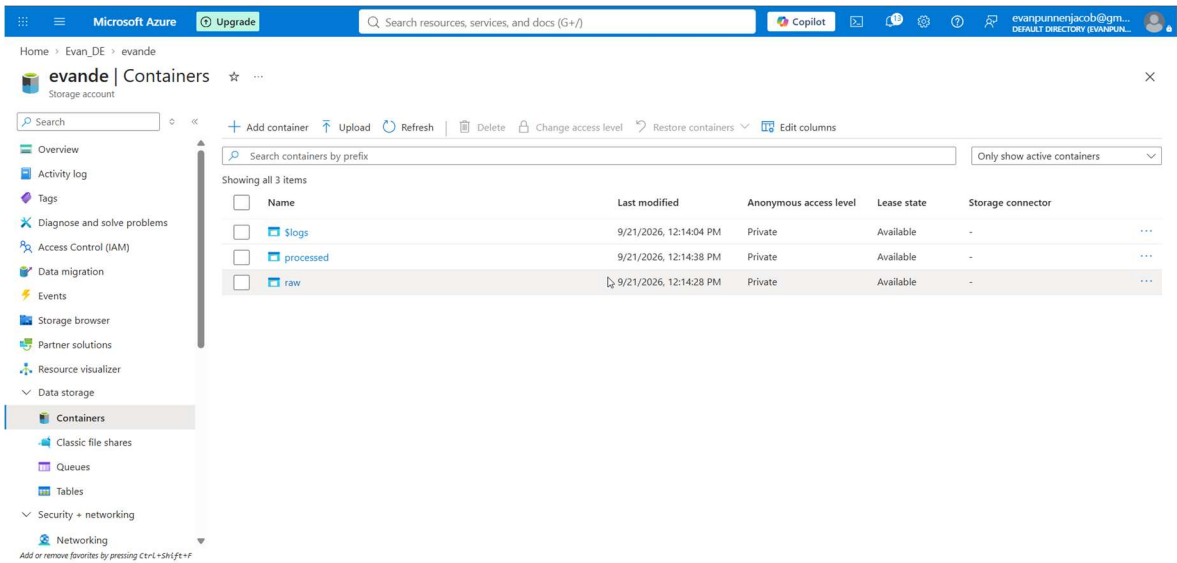
Screenshot evidence 1



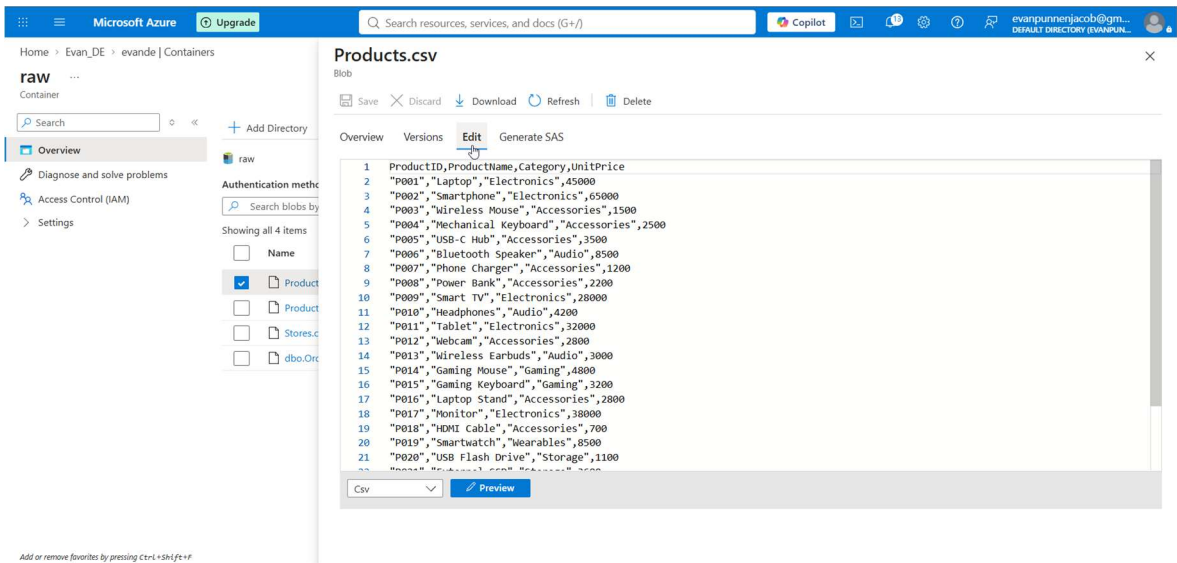
Screenshot evidence 2



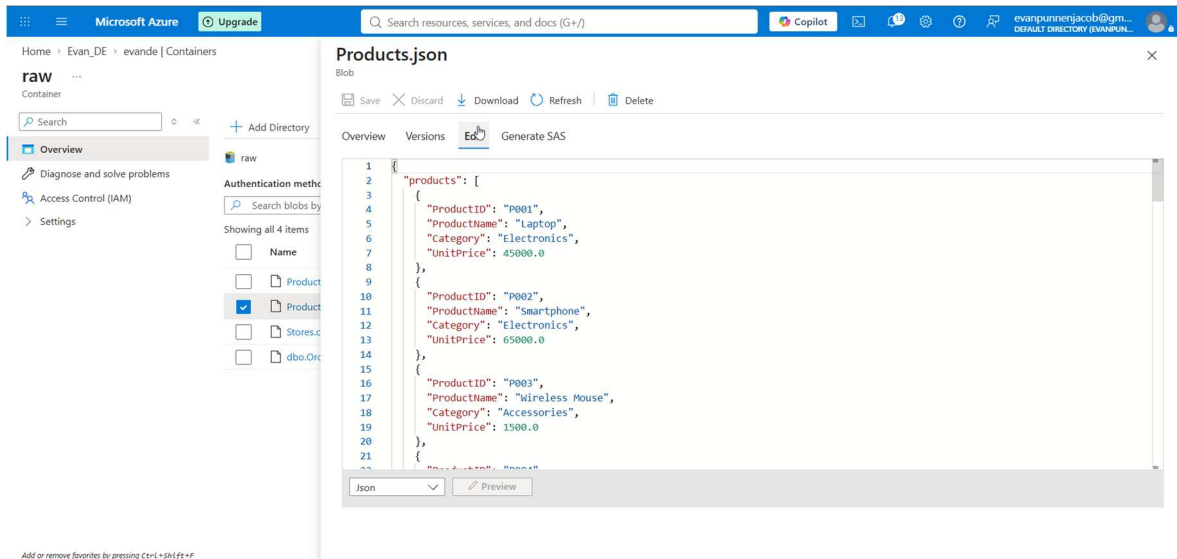
Screenshot evidence 3



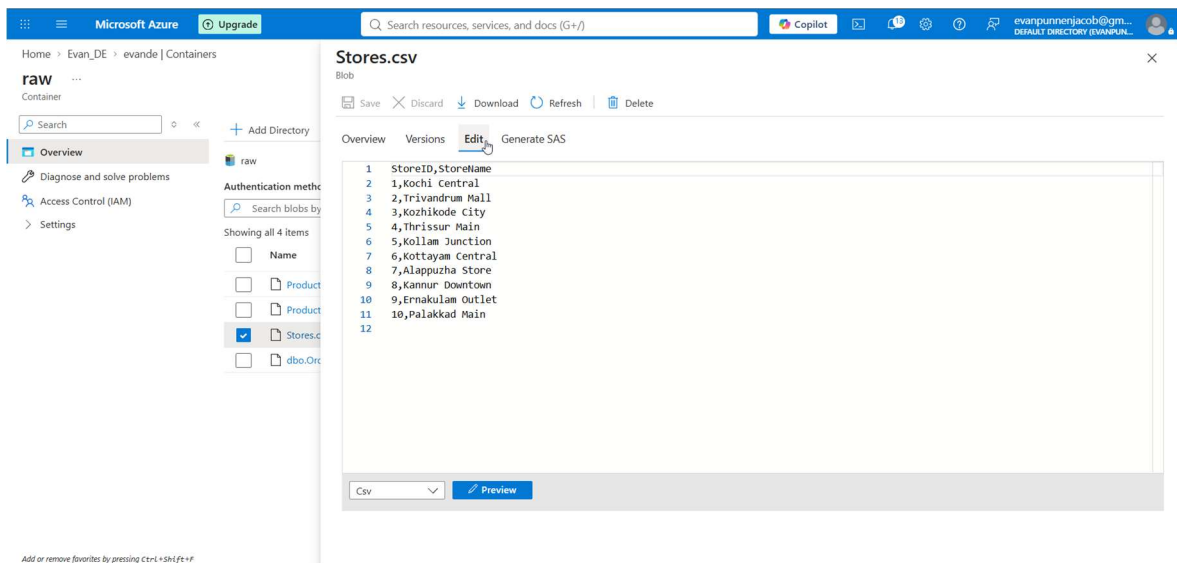
Screenshot evidence 4



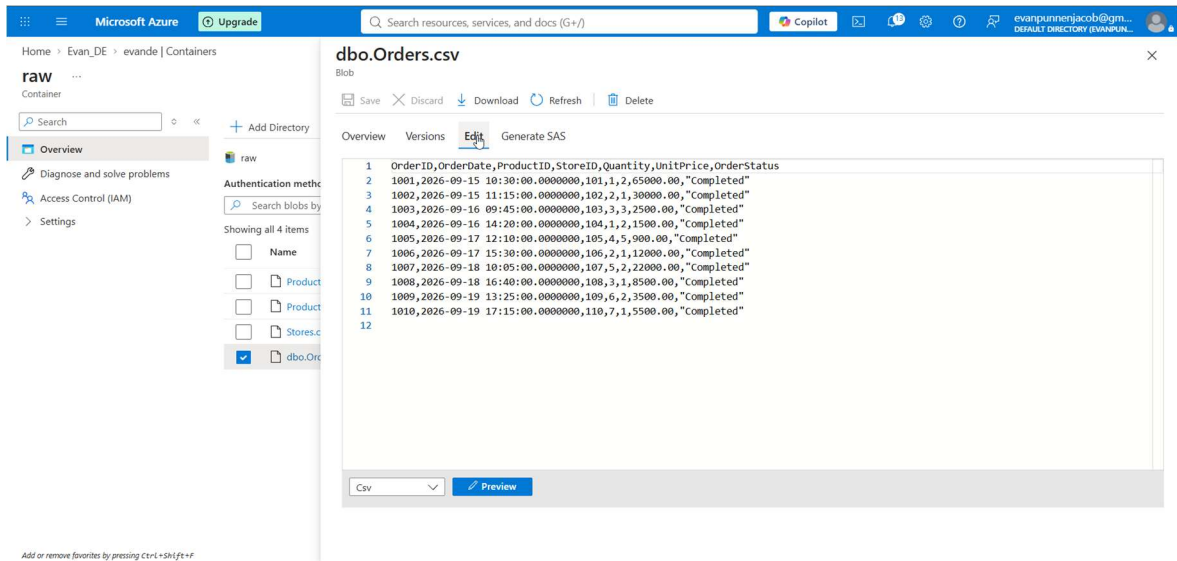
Screenshot evidence 5



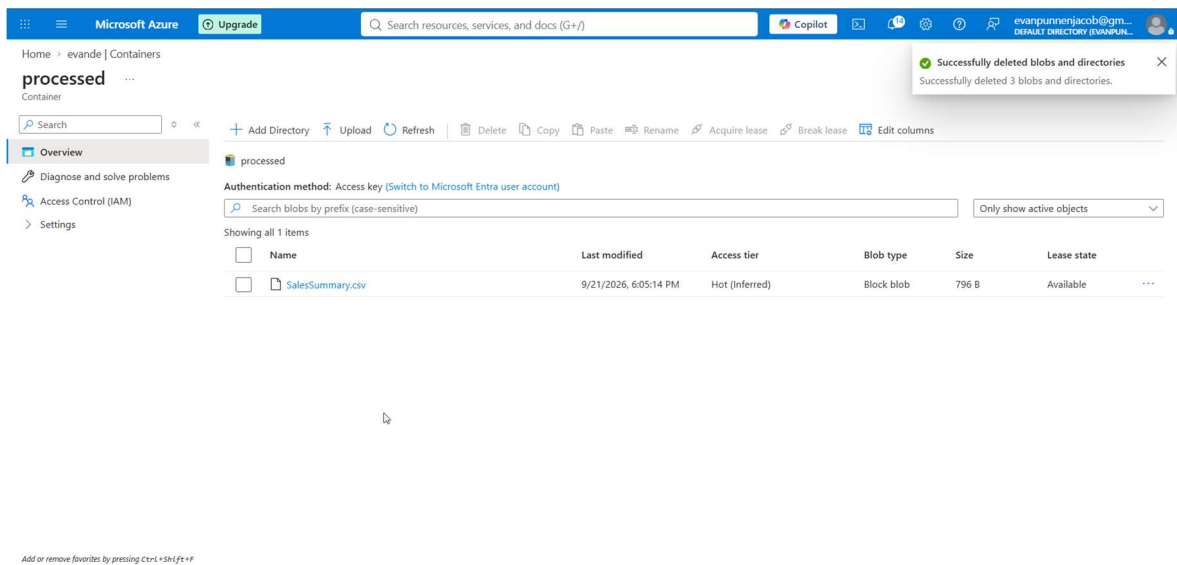
Screenshot evidence 6



Screenshot evidence 7



Screenshot evidence 8



10. Interview Explanation

I built a retail data integration platform using Azure Data Factory. Orders were ingested from Azure SQL, Products from a REST JSON source and Stores from ADLS/Blob storage. I created a raw zone, used Mapping Data Flow for cleansing, Product ID normalization, joins and sales aggregation, and wrote the curated result to a processed CSV. The curated data was loaded into Azure SQL using Upsert with SalesDate, StoreID and ProductID as keys. I added row-count validation, a master orchestrator and a daily schedule trigger.